

Expropriate 题解

V2, 优化了表述, 增加了复杂度分析的章节。

一次操作选中一个元素 x , 将它改成 $\text{inc}(x) = \min(x + 1, m)$, 再插回序列的任意位置。要求恰好操作 K 次, 并使最终序列的字典序最小。下文把数值相同但原位置不同的元素看成不同元素。

整个解法分成两层。先证明多次操作同一个元素时, 只需单独考虑最早出现的全局最大值; 去掉这种情况后, 每个元素都至多操作一次, 再对这个子问题递归。

固定每个元素的操作次数

设第 i 个元素被操作 c_i 次。没有被操作的元素从未离开序列, 因此仍按原下标组成子序列 $U = (a_i \mid c_i = 0)$ 。

只要一个元素至少被操作过一次, 就可以在最后一次操作时放到任意位置。这些元素只需记录最终值的多重集 $B = \{\min(a_i + c_i, m) \mid c_i > 0\}$ 。

固定 (c_i) 后, 最优答案就是 U 与排好序的 B 的稳定归并: 比较 U 的开头与 B 的最小值, 取较小者; 相等时先取 U 。相等时先保留 U 的元素不会改变当前值, 却能更早看到 U 的后续元素, 所以不会更差。记所得序列为 $\text{Merge}(U, B)$ 。

后文说“删除”某个元素, 是指将它操作一次并放入 B , 它仍会出现在最终答案中。

稳定归并还满足下面的单调性: 对于两个等长的局部答案和任意有序多重集 C ,

$$\text{Merge}(U_1, B_1) \leq \text{Merge}(U_2, B_2) \implies \text{Merge}(U_1, B_1 \uplus C) \leq \text{Merge}(U_2, B_2 \uplus C).$$

先注意稳定归并可以分两次进行:

$$\text{Merge}(U, B \uplus C) = \text{Merge}(\text{Merge}(U, B), C).$$

等值时始终先取左边, 所以这个等式也包含相等元素的情况。现在设两个等长序列 $S_1 < S_2$ 的第一个不同位置分别为 $a < b$ 。把它们分别与同一个有序多重集 C 归并时, 小于 a 的公共元素会在两边完全相同地输出。接下来若 C 的最小值小于 a , 两边仍同时输出它; 否则左边先输出 a , 右边只能输出不小于 a 的值。相等只可能是 C 也提供了一个 a , 删去这次相同输出后继续比较即可。由于 S_1, S_2 等长, 不会出现一个序列先结束而反转大小关系。反复进行上述比较即可得到前面的单调性。

本题中同一状态的候选长度相同, 因此递归只需保留子问题自己的最优结果, 不必知道外层之后还会加入哪些相同元素。

多次操作只需考虑全局最大值

先定义两个子问题, 它们都要求每个元素至多操作一次:

$$\begin{aligned} D(A, k) &= \text{恰好选出 } k \text{ 个不同元素, 各操作一次时的最优答案,} \\ E(A, k) &= \min_{0 \leq s \leq k} D(A, s). \end{aligned}$$

E 除了答案, 还记录实际使用的操作数 s 。若不同的 s 得到相同序列, 保留较大的 s 。

考虑一个含有重复操作的方案。先给每个被选中的元素分配一次操作, 剩下的称为额外操作。

设两个已被选中的元素原值为 $x \leq y$, 它们分别得到 $1 + r$ 和 $1 + s$ 次操作, 其中 $r > 0$ 。把 x 的 r 次额外操作全部交给 y , 这两个元素在 B 中的变化是

$$\text{sort}(\min(x + 1 + r, m), \min(y + 1 + s, m)) \longrightarrow \text{sort}(\min(x + 1, m), \min(y + 1 + s + r, m)).$$

后一个二元组的字典序不大于前一个。若 $x < y$ ，新二元组保留了 $x + 1$ ，而旧二元组的两项都不小于它；取等号时两边已经因截断而相同。若 $x = y$ ，忽略截断后，旧二元组较小的一项是 $x + 1 + \min(r, s)$ ，新二元组较小的一项是 $x + 1$ ；只有 $\min(r, s) = 0$ 时可能相等，此时两边较大的一项也相同。截断到 m 只会让这些项提前相等，不会改变方向。 U 没有变化，再由前面的归并单调性，完整答案也不会变差。

不断做这个转移，可以把全部额外操作集中到原值最大的某个已选元素 x 上。于是至多有一个元素被操作多次。

若 x 还不是全局最大值，取一个未被操作的全局最大值 $M > x$ 。这样的 M 一定存在，否则最大的已选元素就应当是 M 。设 x 共被操作 $c \geq 2$ 次。现在让 x 只操作一次，并让 M 操作 $c - 1$ 次。

原方案中， U 含有 M ，而 B 含有 $\min(x + c, m)$ ；新方案从 U 删除 M ，并向 B 放入 $x + 1$ 和 $\min(M + c - 1, m)$ 。

若 $x + 1 < M$ ，在两个方案第一次输出上述元素之一时，新方案可以先输出 $x + 1$ ，而原方案只能输出更大的值。若 $x + 1 = M$ ，则 $\min(M + c - 1, m) = \min(x + c, m)$ ，高值完全相同，区别只在于一个值为 M 的元素从 U 移到了 B 。稳定归并在相等时先取 U ，所以这只会让 U 中后续的不大于 M 的元素更早出现，也不会变差。于是 $c = 2$ 时所有元素都只操作一次； $c > 2$ 时重复操作转移到了全局最大值。

若全局最大值出现多次，还可以把重复操作放在最早出现的那一个上。若它已经被操作一次，只需与当前重复操作的同值元素交换操作次数， U 和 B 都不变；若它尚未操作，则把全部操作次数换到它身上， B 不变，而 U 中保留下来的那个 M 从较早位置移到较晚位置。两者之间的元素都不大于 M ，提前它们不会使答案变差。

于是原问题只剩两个候选：

- K 次操作落在 K 个不同元素上，即 $D(A, K)$ ；
- 最早出现的全局最大值至少操作两次，其余元素至多操作一次。

设这个最大值为 M ，删去它后得到 A^- 。若其余元素使用了 $s \leq K - 2$ 次操作，留给 M 的操作数为 $K - s$ ，它的最终值是 $h_s = \min(M + K - s, m)$ 。

因为 $K - s \geq 2$ ，而其他元素至多增加一次，所以 h_s 是最终序列中的最大值之一，可以直接放在末尾。故

$$\text{Ans}(A, K) = \min(D(A, K), E(A^-, K - 2) \parallel [h_s])$$

第二项只在 $K \geq 2$ 时存在，其中 s 是 $E(A^-, K - 2)$ 实际使用的操作数。当前缀序列相同时， s 越大，留给 M 的操作越少，末尾的 h_s 越小；这就是 E 在平局时保留较大 s 的原因。

每个元素至多操作一次时的递推

先只讨论 D/E 的递推，不考虑数据结构。设 X 是还未决定的序列，并令 $v = \min X$ 、 $w = v + 1$ 。

再定义三个量：

- p ：第一个 v 之前的元素个数；
- c ： X 中 v 的出现次数；
- j ：第一个原有的 w 之前的元素个数；若不存在 w ，令 $j = |X|$ 。

D 必须用完预算 k ， E 只需使用不超过 k 次操作。进入 E 的状态时可以先令 $k \leftarrow \min(k, |X|)$ 。除去边界以及 E 的平局规则，两者使用同一递推。

开头已经是最小值

若 $p = 0$ ，开头的 v 不应被操作。保留它不会消耗预算，而且任何删除它的方案都不可能得到比 v 更小的首项。实现时可以一次保留最长的先导 v 段。

对 E ，整段都可以确定；对 D ，后面必须至少剩下 k 个元素供操作，因此这一轮最多确定 $|X| - k$ 个元素。

预算足以看到第一个最小值

若 $0 < p \leq k$ ，操作第一个 v 之前的 p 个元素，就能让答案以 v 开头。其他方案的首项都大于 v ，所以这个选择是强制的。

记这段前缀为 $P = X[0, p)$ 。递归处理第一个 v 之后的序列，预算变成 $k - p$ ；若子问题返回 (U', B') ，本层返回 $(v \parallel U', \text{inc}(P) \uplus B')$ 。

这里 $\text{inc}(P)$ 表示把 P 中每个元素各增加一次后得到的多重集。

原序列中的 v 和 w 都无法成为首项

现在有 $p > k$ 且 $j > k$ 。预算不足以删去第一个 v 或第一个原有的 w 之前的全部元素，因此当前能够制造出的最小首项只能是 w ，方法是操作一个 v 。

若某个方案操作了较左的 v ，却保留了较右的同值 v ，交换这两个同值元素的选择后， B 不变，而未操作子序列只会更优。因此需要操作若干个 v 时，总可以选择最右侧的那些。

这一轮操作 $q = \min(c, k)$ 个最右侧的 v ，向 B 加入 q 个 w ，再用预算 $k - q$ 递归。若仍有预算，说明所有 v 都已移走，递归中的最小值会自然增大。

第一个原有的 w 可以成为首项

剩下 $p > k$ 且 $j \leq k$ 。第一个原有的 w 位于所有 v 之前，答案只需比较两类方案。

第一类保留这个 w 。操作它之前的 j 个元素后，从它后面继续递归，预算为 $k - j$ 。

第二类让这个 w 无法成为首项。若只操作 $t \leq k - j$ 个最右侧的 v ，在保留 w 的方案中，后缀递归本来就能完成这些操作，不会产生新候选。第一次真正越过这个 w 的阈值是 $q = k - j + 1$ 。

因此只需再考虑：操作最右侧的 q 个 v ，向 B 加入 q 个 w ，再以 $k - q = j - 1$ 为预算递归。继续操作更多 v 的方案已经包含在这个子问题中。若 $q > k$ 或 $c < q$ ，第二类方案不存在。

递推的边界如下：

- $X = \emptyset$ 时返回空序列；
- $k = 0$ 时保留整个 X ；
- $v = m$ 时，继续操作不再改变数值，只更新实际使用的操作数；
- 对 D ，若 $k = |X|$ ，则所有元素各操作一次，全部进入 B 。

四个递推分支分别是 $p = 0$ 、 $0 < p \leq k$ 、 $p > k$ 且 $j > k$ 、 $p > k$ 且 $j \leq k$ ，它们覆盖了所有状态。

递推状态的持久化表示

递推只会删除一段前缀，或删除当前最小值的若干个最右出现，不会产生任意子序列。可以用三个参数表示所有会出现的 X 。

记 $\text{rk}_x(i)$ 为位置 i 在所有值为 x 的位置中的全局编号，从 0 开始。定义

$$X(l, v, r) = (a_i \mid i \geq l, a_i > v \vee (a_i = v \wedge \text{rk}_v(i) < r)).$$

其中 l 表示已经处理完的原序列前缀，小于 v 的值已经全部删除，大于 v 的值仍全部保留，而值为 v 的元素只保留全局前 r 次出现。保留一个元素只会增大 l ，删除最右侧的若干个 v 只会减小 r 。当 v 被删空时，新的最小值此前没有被截断，把 r 重置为它在原序列中的总出现次数即可。

递推所需的操作与维护结构对应如下。

需要的操作	维护方式
求 X 的长度、按秩找原位置、查询区间哈希、寻找第一个大于给定值的元素	按值从大到小建立持久化位置线段树； $\text{ver}[v][r]$ 表示所有大于 v 的位置以及值为 v 的前 r 个位置
找当前最小值，统计或整体转移一段原值	持久化值域频率树 <code>hist</code>
保存未操作子序列 U ，支持区间拼接、最大值和哈希	隐式 Treap
保存有序多重集 B ，支持常值段以及一整张频率表统一增加一次	左偏树维护若干延迟计算的块

位置版本在处理值 v 时先继承所有更大值的位置，再按原顺序加入 v 的各次出现。每个原位置只加入一次，所以这些版本共使用 $O(n \log n)$ 个结点。

`hist` 保存操作前的原值。一段元素统一操作一次时，不必逐个修改，只需把对应的频率树根加入 B ，读取这个块时再统一执行 $x \mapsto \text{inc}(x)$ 。已经确定的最终值则直接保存为常值段。

分割状态时只枚举较短的一边

递推常要在某个原位置处分开 X 。若每次都扫描前缀，复杂度会退化为平方。

每个状态额外缓存分界位置以及左右两边的频率树。分界改变时，只枚举较短的一边，另一边由总频率与旧缓存做持久化加减得到。一个元素每被再次枚举一次，它所在的待分割区间至少缩小一半，因此所有 `splitAt` 合计只枚举 $O(n \log n)$ 个元素。

不展开完整答案也能比较两个候选

递推返回 (U, B, used) ，而不是立刻生成长度为 n 的序列。比较时用一个游标按需执行 $\text{Merge}(U, B)$ 。

若 B 的最小值为 b ，先整段取出 U 中不大于 b 的前缀；遇到 U 中第一个大于 b 的元素后，再取出 B 中所有等于 b 的元素。于是游标只会返回两类块： U 的一个连续区间，或者 B 中值相同的一整段。块有多长并不影响游标前进的次数。

每个块保存模 2^{64} 自然溢出的单项式哈希

$$H(XY) = H(X) \text{BASE}^{|Y|} + H(Y) \pmod{2^{64}}.$$

两个游标同步生成块。当前两块的公共长度可能很长，但只需比较一次区间哈希；哈希相同就整段跳过。若哈希不同，再在这一对块内二分 LCP，并读取第一个不同值。只有最终胜出的答案需要完整展开。

区间哈希用单个 `uint64_t` 维护，隐式 Treap 的优先级由伪随机数生成。

另一种 $O(n \log^3 n)$ 做法

也可以沿着等价的逐字符过程推进，不显式维护上面的 D/E 递归。活动序列的线段树保存两类摘要：删除较大栈顶后的单调栈，以及按前缀最大值分组后的序列表示。每次可以确定下一步应保留原序列元素、取出已操作元素的最小值，还是操作当前最小值；在可能立即停止的位置生成候选并与当前最优答案比较。

候选按最终值 x 分到值域树的叶子。一个叶子写成 $T_x \parallel x^{c_x}$ ，其中 T_x 是该值对应的非平凡前缀，后面接 c_x 个 x 。比较两个候选时，先在值域树上找到第一个指纹不同的叶子，再比较这个叶子中的两段字符串。

这里多出的一个对数出现在第二步。`compareLeaf` 对叶子内部的 LCE 长度二分 $O(\log n)$ 次；每次计算前缀哈希，都要经过表达式树，并在前缀最大值的持久化值域树上查询，保守上界为 $O(\log^2 n)$ 。因此单次叶内 LCE 为 $O(\log^3 n)$ 。其余轨迹维护仍是 $O(n \log^2 n)$ ，总复杂度为 $O(n \log^3 n)$ ，空间复杂度为 $O(n \log^2 n)$ 。

本题 $n \leq 5 \times 10^4$ ，时限为 6 秒，两种做法都能通过。牛客上评测， $O(n \log^2 n)$ 做法约为 400 ms， $O(n \log^3 n)$ 做法约为 3000 ms。

复杂度

先看递归状态数。只有“第一个原有的 w 可以成为首项”这种情况会产生两个分支。设当时第一个 w 的秩为 j ，逃逸分支操作 $q = k - j + 1$ 个 v 。两个子问题的预算分别为 $k - j$ 和 $j - 1$ ，其和为 $(k - j) + (j - 1) = k - 1$ 。

因此每次分支都会让两个孩子的预算总和至少减少 1。其他转移要么减少预算，要么永久确定至少一个元素是否操作。整棵递归树共有 $O(n)$ 个状态，也只会进行 $O(n)$ 次候选比较。

状态分割使用较短一侧枚举。一个元素若在更高层再次被枚举，它所在的待分割区间至少扩大一倍，所以每个元素只会被枚举 $O(\log n)$ 次。每次持久化频率修改需要 $O(\log(m + 1))$ ，所有 `splitAt` 的总时间和新增结点数均为 $O(n \log n \log(m + 1))$ 。

候选比较还要单独做一次摊还。这里计数的是游标生成的块，而不是被哈希跳过的元素个数。

在一个产生两种候选的状态中，保留原有 w 的方案改变了它之前的 j 个元素；另一个方案改变了最右侧的 q 个 v 。比较这两个方案时，每越过一个新的归并边界，都会完整消耗其中一侧的一个 U 区间或一个同值的 B 段。把这次消耗记在造成该边界的较短一组上，本层新增的块数为 $O(\min(j, q) + 1)$ 。

这个收费也解释了两个容易混淆的极端。若 j 或 q 很小，递归可以形成很长的单链，但每层只检查常数个新块；若二者都大，当前比较可能经过很多块，可两个孩子的预算都会明显缩小，之后不可能连续发生很多次。

形式化地，设预算不超过 k 的整棵递归树一共处理 $C(k)$ 个块。两个孩子的预算为 $k - j$ 和 $j - 1$ ，而 $q = k - j + 1$ ，因此

$$C(k) \leq C(k - j) + C(j - 1) + O(\min(j, k - j + 1) + 1).$$

按较小的孩子收费：同一单位下一次再被收费时，包含它的预算范围至少扩大一倍，所以每个单位至多被收费 $O(\log(k + 1))$ 次。由此 $C(k) = O(k \log(k + 1)) = O(n \log n)$ 。

处理一个块需要在隐式 Treap、位置树或优先队列上做 $O(\log n)$ 级别的查询。若当前两块不同，只在这一次比较的最后一对块内二分 LCP，额外花费 $O(\log^2 n)$ ；候选比较共有 $O(n)$ 次。最外层的两个总方案还会单独比较一次，即使它经过 $O(n)$ 个块，也不改变总上界。

结合 $m \leq 2n$ ，前面的 D/E 递推总复杂度为：

- 时间复杂度：期望摊还 $O(n \log^2 n)$ ；

- 空间复杂度： $O(n \log^2 n)$ 。

“期望”来自隐式 Treap 的随机优先级。